COMPUTER ORGANIZATION

Unit-IV: Central Processing Unit

Comprehensive Study Notes

UNIT-IV: CENTRAL PROCESSING UNIT

1.1 What is a CPU?

The **Central Processing Unit (CPU)** is the brain of the computer that executes instructions and processes data. It performs all arithmetic, logical, control, and input/output operations.

1.2 CPU Components

Component	Description
ALU	Performs arithmetic and logical operations
Control Unit	Generates control signals and coordinates operations
Registers	High-speed temporary storage
Cache	Fast memory for frequently accessed data
Internal Bus	Connects CPU components

2.1 What is Register Organization?

Register organization refers to the arrangement and interconnection of registers within the CPU. Registers provide fast temporary storage for data and instructions.

2.2 Common Registers in CPU

Register	Symbol	Size	Function
Accumulator	AC	16-bit	Stores data for ALU operations
Program Counter	PC	12-bit	Holds address of next instruction
Instruction Register	IR	16-bit	Holds current instruction
Memory Address Register	MAR	12-bit	Holds memory address
Memory Data Register	MDR	16-bit	Holds data to/from memory
Index Register	IX	16-bit	Used for indexed addressing
Status Register	SR	8-bit	Holds status flags (Carry, Zero, etc.)

2.3 Control Word

A **control word** is a binary word that specifies the control signals for a particular operation.

Operation	Control Word
ADD R1, R2	001 000 01 10
SUB R1, R2	001 001 01 10
MOVE R1, R2	010 000 01 10
LOAD R1, M[100]	011 000 01 00

STORE R1, M[100]	011 000 01 11

2.4 Micro-operations

Fetch Phase

Step	Micro-operation	Description
T0	MAR ← PC	Transfer PC to MAR
T1	MDR ← M[MAR], PC ← PC + 1	Read memory, increment PC
T2	IR ← MDR	Load instruction into IR

Decode Phase

Step	Micro-operation	Description
T3	D ← Decode IR(opcode)	Decode instruction opcode

Execute Phase for Common Instructions

Instruction	Micro-operation	Description
ADD	AC ← AC + M[address]	Add memory to AC
AND	AC ← AC & M[address]	AND memory with AC
LDA	AC ← M[address]	Load memory into AC
STA	M[address] ← AC	Store AC to memory
BUN	PC ← address	Branch unconditionally
BSA	M[address] ← PC, PC ← address + 1	Branch and save return address

ISZ	M[address] ← M[address] + 1; if zero then PC ← PC + 1	Increment and skip if zero

3.1 What is a Stack?

A **stack** is a data structure that follows the **LIFO (Last In First Out)** principle. It is used for storing temporary data during program execution.

```
+----------------------+  +----------------------+
|    REGISTER STACK    |  |     MEMORY STACK     |
|    (Internal CPU)    |  |        (RAM)         |
+----------------------+  +----------------------+
```

3.2 Register Stack

A **register stack** uses CPU registers to implement a stack.

Operation	Description	RTL Notation
PUSH	Add data to top of stack	SP ← SP + 1, M[SP] ← Data
POP	Remove data from top of stack	Data ← M[SP], SP ← SP - 1

Example:

```
PUSH A: SP ← SP + 1, M[SP] ← A
PUSH B: SP ← SP + 1, M[SP] ← B
POP X:  X ← M[SP], SP ← SP - 1
```

3.3 Memory Stack

A **memory stack** uses a portion of main memory (RAM) for stack operations.

```
PUSH: SP ← SP - 1, M[SP] ← Data
POP:  Data ← M[SP], SP ← SP + 1
```

3.4 Reverse Polish Notation (RPN)

Reverse Polish Notation (RPN) is a mathematical notation where operators follow their operands. It eliminates the need for parentheses and is used in stack-based evaluation.

3.5 Evaluation of Arithmetic Expressions using Stack

Algorithm:

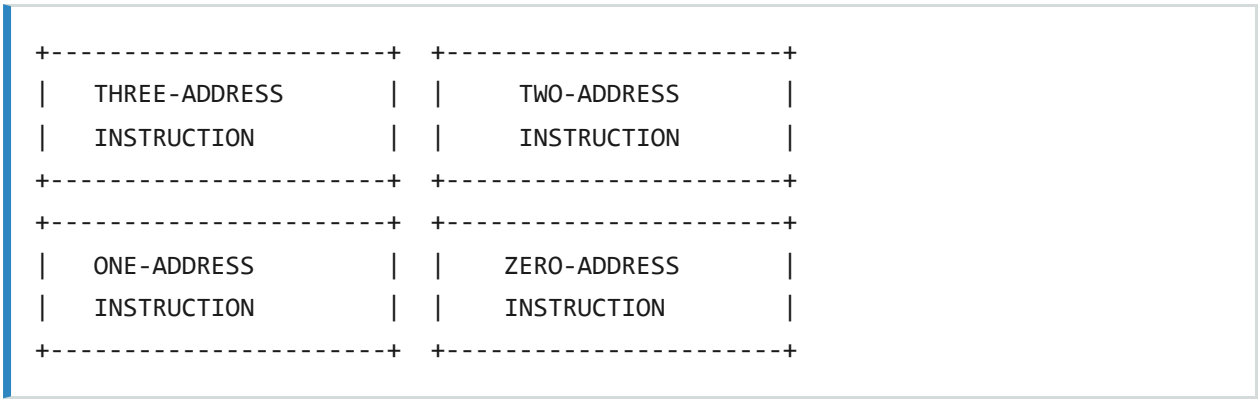
1. Scan expression from left to right
2. If operand, push to stack
3. If operator, pop two operands, perform operation, push result

Example: Evaluate (3 + 4) × 2 in RPN: 3 4 + 2 ×

Step	Symbol	Stack Contents	Operation
1	3	3	Push 3
2	4	3, 4	Push 4
3	+	7	Pop 4, Pop 3, Push 3+4=7
4	2	7, 2	Push 2
5	×	14	Pop 2, Pop 7, Push 7×2=14

4.1 What are Instruction Formats?

Instruction formats define how instructions are arranged in memory. They specify the number of operands and how they are addressed.



4.2 Three-Address Instructions

Three-address instructions have three operands: two source operands and one destination.

Instruction	Operation	Meaning
ADD R1, R2, R3	R1 ← R2 + R3	Add R2 and R3, store in R1
SUB R1, R2, R3	R1 ← R2 - R3	Subtract R3 from R2, store in R1
MUL R1, R2, R3	R1 ← R2 * R3	Multiply R2 and R3, store in R1
DIV R1, R2, R3	R1 ← R2 / R3	Divide R2 by R3, store in R1

4.3 Two-Address Instructions

Two-address instructions have two operands: one source and one destination.

Instruction	Operation	Meaning
ADD R1, R2	R1 ← R1 + R2	Add R2 to R1

SUB R1, R2	R1 ← R1 - R2	Subtract R2 from R1
MUL R1, R2	R1 ← R1 * R2	Multiply R1 by R2
MOV R1, R2	R1 ← R2	Copy R2 to R1

4.4 One-Address Instructions

One-address instructions use the accumulator (AC) as an implicit operand.

Instruction	Operation	Meaning
LOAD R1	AC ← R1	Load R1 into AC
STORE R1	R1 ← AC	Store AC to R1
ADD R1	AC ← AC + R1	Add R1 to AC
SUB R1	AC ← AC - R1	Subtract R1 from AC
MUL R1	AC ← AC * R1	Multiply AC by R1

4.5 Zero-Address Instructions (Stack Instructions)

Zero-address instructions use a stack implicitly.

Instruction	Operation
PUSH X	Stack ← X
POP X	X ← Stack
ADD	Top ← Top + Second
SUB	Top ← Top - Second
MUL	Top ← Top * Second

4.6 RISC vs CISC

Feature	RISC	CISC
Instruction Set	Small, simple instructions	Large, complex instructions
Instruction Format	Fixed length (usually 32-bit)	Variable length
Addressing Modes	Few (usually 1-2)	Many (12+)
Registers	Many (32-128 general purpose)	Few (8-16 general purpose)
Control Unit	Hardwired (faster)	Microprogrammed (slower)
Execution Time	Most instructions execute in 1 clock cycle	Instructions take multiple clock cycles
Memory Access	Only LOAD/STORE access memory	Many instructions can access memory
Pipelining	Easier to pipeline	Harder to pipeline
Examples	ARM, MIPS, RISC-V	x86, 68000, VAX

4.7 Pipelining

Pipelining is a technique where multiple instructions are overlapped in execution, similar to an assembly line.

```
Without Pipelining (Sequential):
+--------+ +--------+ +--------+
| Instr1 | | Instr2 | | Instr3 |
+--------+ +--------+ +--------+

With Pipelining (Overlapped):
+----+----+----+----+
| F1 | D1 | E1 |    |
+----+----+----+----+
```

```
    | F2 | D2 | E2 |
    +----+----+----+
         | F3 | D3 | E3 |
         +----+----+----+
```

Stage	Description
Fetch (F)	Fetch instruction from memory
Decode (D)	Decode instruction and read registers
Execute (E)	Perform ALU operation or address calculation
Memory (M)	Access memory if needed
Write-back (W)	Write result to register

Speedup: Ideal speedup = Number of stages. A 5-stage pipeline can ideally execute instructions 5 times faster than sequential execution.

5.1 Addressing Modes (from Unit IV perspective)

Mode	Effective Address	Example	Advantages	Disadvantages
Immediate	Operand value	ADD #5	Fast	Limited value
Direct	Address from instruction	LOAD 1000	Simple	Fixed address
Indirect	M[Address from instruction]	LOAD (1000)	Flexible	Slower
Register	Register value	ADD R1	Fast	Limited registers
Indexed	Address + Index	LOAD 1000(X)	Flexible	Complex
Relative	PC + Offset	JMP +5	Good for loops	Complex

6.1 Data Transfer Instructions

Data transfer instructions move data between registers, memory, and I/O devices.

6.2 Data Manipulation Instructions

6.2.1 Arithmetic Instructions

6.2.2 Logical and Bit Manipulation Instructions

6.2.3 Shift Instructions

7.1 Status Bit Conditions

Status bits (or flags) indicate the result of arithmetic and logical operations.

Flag	Symbol	Description
Carry	C	Set if carry out of MSB
Zero	Z	Set if result is zero
Sign	S	Set if MSB is 1 (negative)
Overflow	V	Set if overflow occurs
Parity	P	Set if even number of 1s

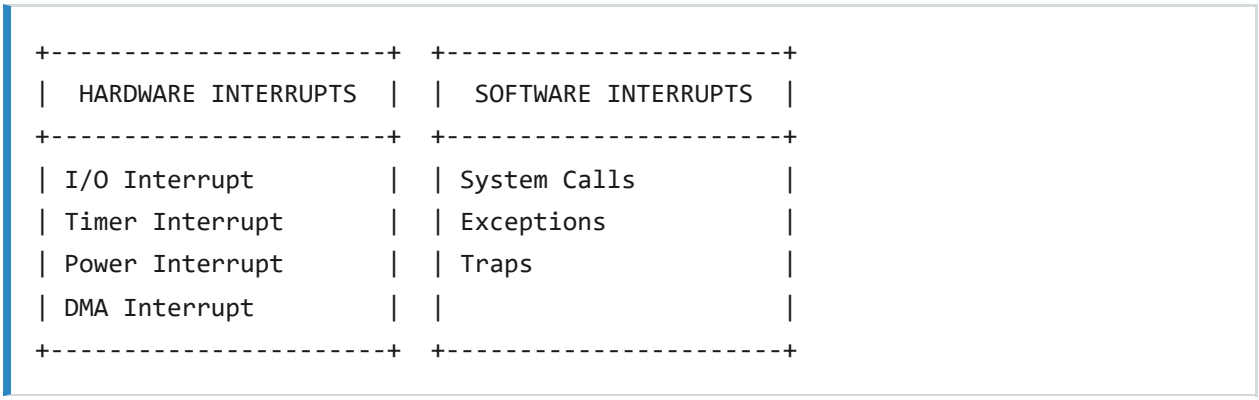
7.2 Conditional Branch Instructions

Conditional branch instructions change the program flow based on status flags.

7.3 Program Interrupt

Program interrupt is a mechanism that temporarily stops the current program to handle an event.

7.4 Types of Interrupts



Questions

Short Answer Questions (2-5 Marks)

1. What are the components of a CPU?
2. What is a control word?
3. What is a stack?
4. What is Reverse Polish Notation?
5. What are instruction formats?
6. What is the difference between direct and indirect addressing?
7. What are status bits?
8. What is a conditional branch?
9. What is program interrupt?
10. What are the types of interrupts?

Long Answer Questions (10 Marks)

1. Explain the general register organization with a diagram.
2. Explain stack organization and its operations.
3. Explain Reverse Polish Notation and stack evaluation.
4. Explain different instruction formats with examples.
5. Explain addressing modes with examples.
6. Explain data transfer and manipulation instructions.
7. Explain status bits and conditional branch instructions.
8. Explain program interrupt and its types.

Objective Questions (1 Mark)

1. The CPU consists of:
   (a) ALU (b) Control Unit (c) Registers (d) All
2. LIFO stands for:
   (a) Last In First Out (b) First In First Out (c) Both (d) None
3. RPN stands for:
   (a) Reverse Polish Notation (b) Regular Polish Notation (c) Both (d) None

4. In immediate addressing, the operand is:
   (a) In memory (b) In instruction (c) In register (d) None
5. Z flag indicates:
   (a) Zero result (b) Carry (c) Overflow (d) Sign

QUICK REVISION

CPU Components

Stack Operations

PUSH: SP ← SP - 1, M[SP] ← Data POP: Data ← M[SP], SP ← SP + 1

Instruction Formats

3-Address: OPCODE, R1, R2, R3 2-Address: OPCODE, R1, R2 1-Address: OPCODE, R1 0-Address: OPCODE

Addressing Modes

Immediate: Operand in instruction Direct: Address in instruction

Indirect: Address contains address Register: Register operand

Indexed: Address + Index Relative: PC + Offset

End of Unit-IV Notes